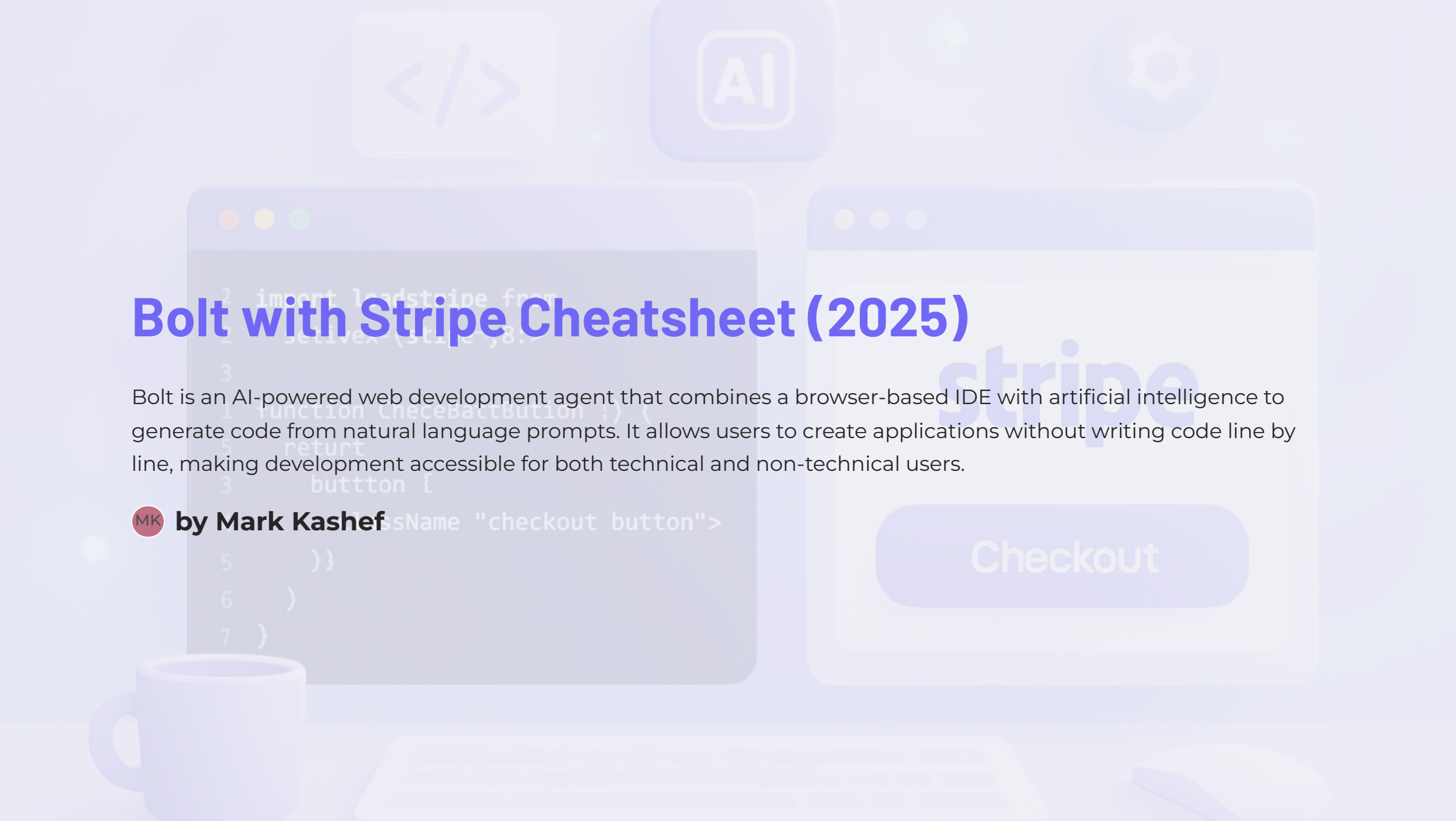
Bolt with Stripe Cheatsheet (2025)

Bolt is an AI-powered web development agent that combines a browser-based IDE with artificial intelligence to generate code from natural language prompts. It allows users to create applications without writing code line by line, making development accessible for both technical and non-technical users.

MK **by Mark Kashef**



Core Technology

AI Engine

Powered by Anthropic's Claude 3.5 Sonnet LLM

Development Environment

Browser-based IDE using StackBlitz's WebContainers technology

Backend Integration

Native Supabase integration for database functionality

Payment Processing

Built-in Stripe integration for payments and subscriptions

Deployment

One-click deployment to Netlify

Key Features at a Glance

Browser-based IDE

No local setup required

AI-powered code generation

Create code from natural language

Native Stripe integration

Built-in payment processing

Figma integration

Design-to-code conversion

Native mobile support

Via Expo integration

One-click deployment

Easy publishing to Netlify

Key Strengths

Rapid Prototyping

Create functional applications quickly from natural language descriptions

No Coding Required

Generate working code without programming knowledge

Cost-Effective

Significantly reduce development costs compared to traditional methods

Iterative Development

Easily refine and improve applications as you go

Stripe Integration

Seamless payment processing capabilities

Deployment Simplicity

One-click deployment to Netlify with custom domain support

Limitations

Token Consumption

Can become expensive for complex projects

Debugging Challenges

Limited capabilities for resolving complex issues

UI Customization

Complex UI modifications often require manual intervention

Error Handling

Struggles with architectural and deep structural problems

Backend Requirements

Requires Supabase for Stripe integration (Firebase not supported)

Scaling Issues

Component duplication and inconsistency in larger projects

Natural Language Prompts

The primary way to interact with Bolt is through natural language prompts that describe what you want to build.

Best Practices:

- Be as specific as possible about features and requirements
- Break down complex applications into manageable components
- Use clear, descriptive language about functionality and design
- Reference existing UI patterns when applicable

Example Prompts:

> "Create an e-commerce site with product listings, shopping cart, and Stripe checkout integration. The design should be modern with a dark theme."

> "Build a membership site with tiered subscription plans using Stripe. Include user authentication, profile management, and content access control based on subscription level."

> "Design a recipe tracking app where users can add, edit, and categorize recipes. Include a search function and filtering by ingredients."

Enhance Prompt Feature

Use the enhance prompt feature to add more detail to your initial description.

How to Use:

1. Start with a basic prompt describing your application
2. Use the enhance prompt feature to add specific details
3. Review the enhanced prompt and make any necessary adjustments
4. Submit the final prompt for generation

Example Enhancement:

Initial: "Create a recipe tracking app"

Enhanced: "Create a recipe tracking app with the following features: - User authentication with email and password - Ability to add, edit, and delete recipes - Recipe categorization by meal type and cuisine - Search functionality by recipe name and ingredients - Responsive design for mobile and desktop - Option to mark favorites and create collections"

Interface Navigation

Code Editor

- Central area for viewing and editing generated code
- Syntax highlighting and auto-completion
- File navigation panel on the left side

Preview Panel

- Real-time preview of your application
- Updates as code changes are made
- Responsive testing options

Chat Interface

- Area to input prompts and receive responses
- History of previous interactions
- Options to refine or modify requests

Terminal

- Run commands directly in the browser
- Install packages and dependencies
- Execute scripts and tests

Project Setup

Start a new project

- Navigate to Bolt dashboard
- Click "New Project"
- Choose a template or start from scratch
- Name your project

Define project requirements

- Write a detailed prompt describing your application
- Use the enhance prompt feature to add specifics
- Include information about design, functionality, and data requirements

Initial generation

- Submit your prompt to generate the initial codebase
- Review the generated code and application structure
- Test the basic functionality in the preview panel

Stripe Integration Workflow

1

Set up Supabase

- Ensure Supabase is connected to your project
- Set up authentication (required for Stripe integration)
- Configure database tables for user and payment data

Configure Stripe

- Add Stripe dependency to your project
- Create Stripe initialization file with API keys
- Set up products and prices in Stripe dashboard

Implement payment flows

- Create checkout components for one-time payments
- Set up subscription management for recurring payments
- Configure webhook handling for event processing

Test payment processing

- Use Stripe test cards in sandbox environment
- Verify successful payment processing
- Test subscription creation and management
- Validate webhook event handling

Iterative Development

Review and refine

- Identify areas for improvement
- Create specific prompts for enhancements
- Use the Target file feature to focus changes on specific files

Lock critical files

- Identify files that should not be modified
- Use the Lock files feature to prevent unwanted changes
- Maintain custom code while allowing AI to modify other areas

Switch perspectives

- Toggle between diff and original feature previews
- Compare changes before applying them
- Make informed decisions about modifications

Token Conservation Strategies

- Enable the "diffs" feature to save tokens during changes
- Focus on one feature or component at a time
- Use Bolt for initial framework and major features, then switch to traditional coding for details
- Lock files that don't need modification to prevent token usage on unchanged files
- Plan your development approach before starting to minimize iterations

Token Efficiency

Enable diffs feature

Prevents rewriting entire files during small changes

Target specific files

Focus changes on relevant files only

Lock stable files

Prevent modifications to files that are working correctly

Batch related changes

Combine similar modifications into single prompts

Plan before prompting

Think through requirements to minimize iterations

Application Speed

Minimize dependencies

Request streamlined package usage

Code splitting

Ask for code splitting implementation for larger applications

Lazy loading

Implement for components not immediately needed

Image optimization

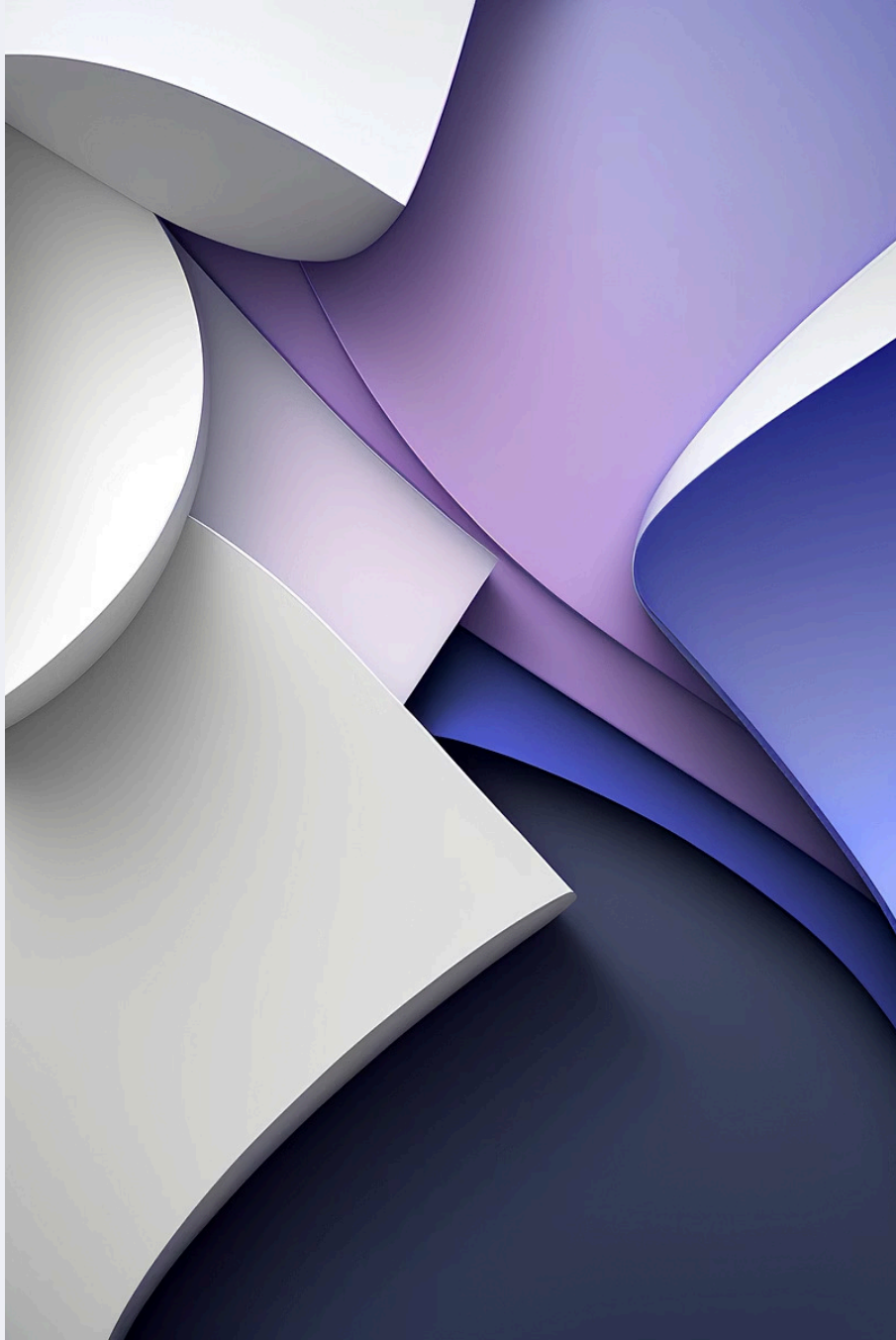
Use responsive images and proper formats

Caching strategies

Implement appropriate caching for data and assets

Stripe Performance

- Optimize checkout flow: Streamline the payment process
- Implement proper error handling: Add comprehensive validation
- Use Stripe Elements: Leverage optimized payment components
- Configure webhook processing: Ensure efficient event handling
- Implement idempotency keys: Prevent duplicate processing



Debugging Strategies

Identify error patterns

Look for recurring issues

Rewrite problematic sections

Sometimes starting fresh is more efficient

Use console logging

Add strategic logging for troubleshooting

Implement error boundaries

Contain failures to specific components

Test incrementally

Validate changes in small batches

Stripe Integration

Payment Types

One-time payments and subscriptions

Setup Requirements

Supabase connection and authentication

Implementation

Edge functions for secure processing

Products Management

Automatic sync from Stripe dashboard

Testing

Sandbox environment with test cards

Security

Secure handling of API keys and webhook signatures

Example Stripe Checkout Implementation

```
// Initialize Stripe with your API key
const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);

// Create a checkout session for one-time payment
async function createCheckoutSession(req, res) {
  try {
    const session = await stripe.checkout.sessions.create({
      payment_method_types: ['card'],
      line_items: [{
        price: 'price_1234567890',
        quantity: 1,
      }],
      mode: 'payment',
      success_url: `${process.env.DOMAIN}/success?session_id=
{CHECKOUT_SESSION_ID}`,
      cancel_url: `${process.env.DOMAIN}/cancel`,
    });
    res.json({ id: session.id });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
}
```

```
const stripe = await
loadStripe('pk_test_ABC1123')

const handleCheckout() =
 async stripe =
  stripe.redirectToCheckout({
   lineItems: [
     price: 'prrice_123',
     quantity: 1
   ],
   mode: 'payment',
   successuri:
    'https://example.com/success'
  }
 )
)
```

Figma Integration

Design Import

Convert Figma designs to code

Asset Extraction

Automatically extract images and icons

Style Mapping

Convert design tokens to CSS variables

Component Creation

Generate React components from Figma elements

Responsive Adaptation

Implement responsive behavior based on design

Expo Integration for Mobile

- **Native Mobile Support:** Create iOS and Android applications
- **Component Adaptation:** Automatically adapt web components to native
- **Navigation Implementation:** Set up mobile-friendly navigation patterns
- **Device Feature Access:** Utilize device capabilities like camera and location
- **Testing:** Preview on physical devices or emulators

Netlify Deployment

One-Click Deployment

Streamlined publishing process

Custom Domains

Connect your own domain

SSL Certificates

Automatic HTTPS setup

CI/CD Integration

Continuous deployment from repository

Environment Variables

Secure configuration management

Best For: E-commerce Applications

- Product catalogs with Stripe checkout
- Subscription-based services
- Membership sites with tiered access
- Digital product delivery

Optimization Tips:

- Start with payment flow implementation early
- Use Stripe Elements for optimized checkout
- Implement proper inventory management
- Focus on secure user authentication
- Test thoroughly with various payment scenarios

Best For: MVPs and Prototypes

- Proof-of-concept applications
- Investor demonstrations
- User testing platforms
- Feature validation

Optimization Tips:

- Focus on core functionality first
- Minimize custom UI elements initially
- Implement analytics to track user behavior
- Create a clear path to validate key hypotheses
- Design with future scaling in mind

Best For: Content-Focused Websites

- Blogs and publishing platforms
- Portfolio websites
- Marketing landing pages
- Documentation sites

Optimization Tips:

- Prioritize content management workflows
- Implement efficient image handling
- Focus on SEO-friendly structure
- Create responsive layouts for all devices
- Optimize loading performance

Best For: Basic CRUD Applications

- Inventory management
- Simple CMS systems
- Data collection forms
- Record keeping applications

Optimization Tips:

- Design database schema carefully
- Implement proper validation and error handling
- Create intuitive data entry forms
- Focus on efficient data retrieval patterns
- Implement appropriate access controls

Best For: Mobile Applications (via Expo)

- Cross-platform mobile apps
- Progressive web applications
- Hybrid web/mobile experiences

Optimization Tips:

- Use Expo-compatible components
- Test on multiple device sizes
- Implement touch-friendly interfaces
- Consider offline functionality
- Optimize for mobile network conditions

Common Issues and Solutions

Token Consumption Issues

Symptoms: Rapidly depleting token allowance, unexpected token usage

Solutions:

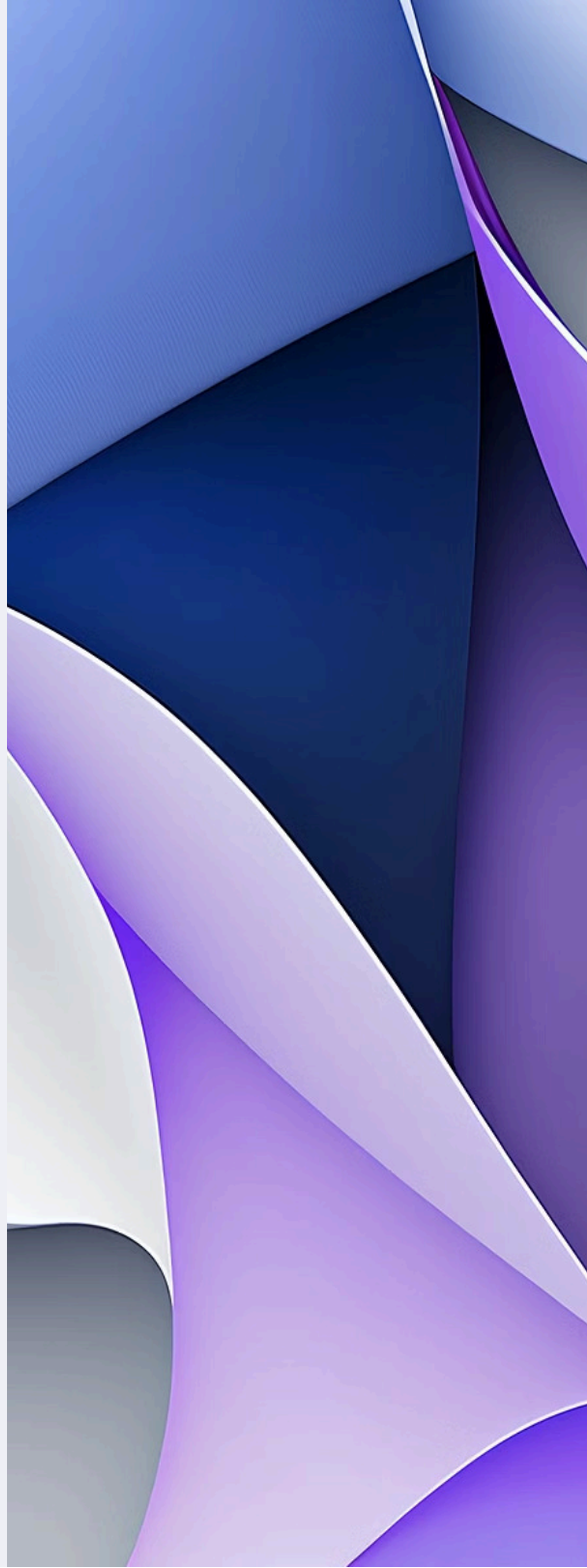
- Enable the "diffs" feature in settings
- Lock files that don't need modification
- Target specific files for changes
- Use more specific, focused prompts
- Consider upgrading to a higher token plan

Stripe Integration Problems

Symptoms: Payment processing failures, webhook errors, subscription issues

Solutions:

- Verify Supabase authentication is properly configured
- Check Stripe API keys (test vs. live)
- Ensure webhook endpoints are correctly set up
- Validate product and price configurations in Stripe dashboard
- Implement proper error handling and logging



More Common Issues and Solutions

Code Generation Quality

Symptoms: Inconsistent code, missing functionality, structural issues

Solutions:

- Provide more detailed prompts with specific requirements
- Break complex requests into smaller, focused prompts
- Use examples to illustrate desired patterns
- Manually edit problematic sections
- Lock well-functioning files to prevent regression

Deployment Failures

Symptoms: Build errors, deployment timeouts, runtime issues

Solutions:

- Check for missing dependencies
- Verify environment variables are properly set
- Review build logs for specific errors
- Test locally before deployment
- Consider incremental deployments for large applications

Debugging Workflow

Identify the issue

Determine the specific problem and its scope

Isolate the cause

Narrow down to specific files or components

Create a targeted prompt

Ask for specific fixes or improvements

Review generated changes

Carefully examine proposed modifications

Test thoroughly

Verify the issue is resolved across different scenarios

Document the solution

Note what worked for future reference

Quick Reference

Token Usage Guidelines

- 1 token ≈ 4 characters of text (English)
- Complex applications require more tokens
- UI changes can consume substantial tokens
- Enable "diffs" feature to save tokens during changes
- Monthly tokens expire; purchased reloads carry forward

Stripe Integration Checklist

- Supabase connection established
- Authentication implemented
- Stripe dependency added
- API keys configured
- Products set up in Stripe dashboard
- Checkout components implemented
- Webhook handling configured
- Payment flows tested with test cards

Plan Comparison

Feature	Pro ($20/mo)	Pro 50 ($50/mo)	Pro 100 ($100/mo)	Pro 200 ($200/mo)
Monthly Tokens	10 million	26 million	55 million	120 million
Token Reloads	Available	Available	Available	Available
Expiration	Monthly	Monthly	Monthly	Monthly
Rollover	No	No	No	No

Support Resources

- Documentation: **docs.bolt.new**
- Community Forum: **forum.bolt.new**
- Stripe Documentation: **stripe.com/docs**
- Supabase Documentation: **supabase.com/docs**
- Netlify Documentation: docs.netlify.com